

De la ecuación del movimiento a una simulación con Euler en Python y Pygame

Ing. Irving Cardona

23 de abril de 2026

**modelo físico $\longrightarrow$ ecuación $\longrightarrow$ discretización $\longrightarrow$ algoritmo $\longrightarrow$
simulación visual**

¿Qué representa una partícula?

Una partícula es una versión simplificada de un objeto físico. En una simulación, normalmente guardamos su **estado**:

posición $\vec{r}(t) = (x(t), y(t))$

velocidad $\vec{v}(t) = (v_x(t), v_y(t))$

aceleración $\vec{a}(t) = (a_x(t), a_y(t))$

¿Por qué guardar estas variables?

Porque el movimiento no se decide “de una vez”; se actualiza con el tiempo. Si sabemos posición, velocidad y aceleración en un instante, podemos aproximar el siguiente paso.

En física, el tiempo es continuo. En la computadora, trabajamos por **pasos**.

$$t_0, t_1 = t_0 + h, t_2 = t_0 + 2h, \dots$$

- h se llama **paso temporal**.
- La computadora no “ve” la curva completa: solo calcula una secuencia de estados.
- Entre más pequeño sea h , mejor suele ser la aproximación, pero aumenta el costo computacional.

¿Por qué discretizamos?

Porque la ecuación diferencial describe cómo cambia el sistema, pero la computadora necesita instrucciones finitas y repetibles frame por frame.

Ecuación de la gravedad en caída vertical

Si una partícula solo se mueve verticalmente bajo gravedad, la ecuación física es:

$$\frac{d^2y}{dt^2} = -g$$

- $y(t)$ es la posición vertical.
- g es la aceleración gravitatoria.
- El signo negativo indica que la aceleración apunta hacia abajo si tomamos hacia arriba como positivo.

¿Qué nos dice esta ecuación?

No nos da directamente la posición del siguiente frame. Solo nos dice la **tasa de cambio de la velocidad**. Por eso hay que transformarla en un esquema numérico.

Paso 1: convertir una ecuación de segundo orden en un sistema de primer orden

Euler trabaja más cómodamente con ecuaciones de primer orden. Por eso introducimos la velocidad:

$$v(t) = \frac{dy}{dt}$$

Entonces el problema queda como:

$$\begin{aligned}\frac{dy}{dt} &= v(t) \\ \frac{dv}{dt} &= -g\end{aligned}$$

¿Por qué se hace esto?

Porque ahora tenemos dos reglas simples:

- una para actualizar la posición,
- otra para actualizar la velocidad.

Paso 2: aproximar derivadas con diferencias finitas

La idea de Euler es reemplazar derivadas por cambios pequeños:

$$\frac{dy}{dt} \approx \frac{y_{n+1} - y_n}{h}$$
$$\frac{dv}{dt} \approx \frac{v_{n+1} - v_n}{h}$$

Sustituyendo en el sistema:

$$\frac{y_{n+1} - y_n}{h} \approx v_n$$
$$\frac{v_{n+1} - v_n}{h} \approx -g$$

¿Por qué usar v_n y no v_{n+1} ?

En **Euler explícito** usamos la información disponible al inicio del paso. Eso lo hace sencillo de programar.

Paso 3: despejar las fórmulas de Euler explícito

Despejando las ecuaciones anteriores obtenemos:

$$y_{n+1} = y_n + h v_n$$

$$v_{n+1} = v_n - h g$$

Interpretación física

- **Posición nueva:** posición actual + lo que avanzó durante el paso.
- **Velocidad nueva:** velocidad actual + efecto acumulado de la aceleración.

Estas dos ecuaciones ya son un algoritmo.

En dos dimensiones trabajamos con vectores:

$$\vec{r}_{n+1} = \vec{r}_n + h \vec{v}_n$$

$$\vec{v}_{n+1} = \vec{v}_n + h \vec{a}_n$$

Si solo hay gravedad:

$$\vec{a}_n = (0, g_{\text{pantalla}})$$

Nota práctica importante

En una ventana de computadora, el eje y suele crecer **hacia abajo**. Por eso en Pygame muchas veces la gravedad aparece como un valor positivo en y .

Pseudocódigo

Si ya conocemos $\vec{r}$, $\vec{v}$ y $\vec{a}$, cada frame hacemos:

```
# Euler explícito
```

```
velocidad = velocidad + aceleracion * h
```

```
posicion = posicion + velocidad_anterior * h
```

Una forma equivalente y muy usada es:

```
posicion = posicion + velocidad * h
```

```
velocidad = velocidad + aceleracion * h
```

Conviene fijar desde el inicio:

- qué variables guarda cada partícula,
- cuál es el valor de h ,
- en qué orden se actualizan posición y velocidad.

Ejemplo numérico completo

Supongamos:

$$y_0 = 100$$

$$v_0 = 0$$

$$g = 9.8$$

$$h = 0.1$$

Aplicamos Euler explícito:

$$v_1 = v_0 - hg = 0 - 0.1(9.8) = -0.98$$

$$y_1 = y_0 + hv_0 = 100 + 0.1(0) = 100$$

Siguiente paso:

$$v_2 = v_1 - hg = -0.98 - 0.98 = -1.96$$

$$y_2 = y_1 + hv_1 = 100 + 0.1(-0.98) = 99.902$$

¿Qué observamos?

Al inicio la posición no cambia porque la velocidad inicial era cero. Después la velocidad acumulada hace que la partícula empiece a caer.

Una partícula se mueve horizontalmente sin aceleración.

$$\frac{d^2x}{dt^2} = 0$$

Actividad:

- 1 Convierte la ecuación a sistema de primer orden.
- 2 Escribe las fórmulas de Euler para x y v_x .
- 3 Si $x_0 = 20$, $v_{x,0} = 5$ y $h = 0.2$, calcula x_1 y $v_{x,1}$.

Pista: si no hay aceleración, la velocidad no cambia.

Solución del ejercicio guiado 1

Sistema de primer orden:

$$\begin{aligned}\frac{dx}{dt} &= v_x \\ \frac{dv_x}{dt} &= 0\end{aligned}$$

Euler explícito:

$$\begin{aligned}x_{n+1} &= x_n + h v_{x,n} \\ v_{x,n+1} &= v_{x,n}\end{aligned}$$

Con los datos:

$$\begin{aligned}v_{x,1} &= 5 \\ x_1 &= 20 + 0.2(5) = 21\end{aligned}$$

De Euler a una clase en Python

Una implementación mínima con Euler explícito podría verse así:

```
class Particula:
    def __init__(self, x, y, vx=0, vy=0):
        self.pos = pygame.Vector2(x, y)
        self.vel = pygame.Vector2(vx, vy)
        self.acc = pygame.Vector2(0, 0.5)

    def actualizar(self, h=1):
        self.pos += self.vel * h
        self.vel += self.acc * h
```

¿Qué representa cada línea?

- pos: la variable $\vec{r}_n$.
- vel: la variable $\vec{v}_n$.
- acc: la fuerza neta expresada como aceleración.
- actualizar: una iteración del método de Euler.

¿Dónde entra Pygame?

Pygame no resuelve la física. Pygame:

- crea la ventana,
- lee el mouse y el teclado,
- dibuja partículas y ligaduras,
- repite el ciclo visual.

La estructura típica es:

- 1 leer eventos,
- 2 actualizar física,
- 3 corregir restricciones,
- 4 dibujar,
- 5 mostrar el frame.

Idea clave

El método numérico es el motor matemático. **Pygame** es la interfaz visual e interactiva.

¿Y si queremos un péndulo o una cuerda?

Con una partícula sola podemos estudiar movimiento. Para construir estructuras, necesitamos **restricciones o ligaduras**.

Si dos partículas deben mantenerse a una distancia fija L , medimos primero la distancia actual:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Luego corregimos si $d \neq L$.

¿Por qué se corrige después de actualizar?

Porque primero dejamos que la física mueva el sistema, y después reparamos la geometría que debe conservarse.

Corrección geométrica de una ligadura

La idea es repartir el error entre ambas partículas:

$$\Delta d = L - d$$
$$\text{corrección} \propto \frac{\Delta d}{2} \frac{\vec{p}_2 - \vec{p}_1}{d}$$

- Si la distancia actual es mayor que la deseada, las partículas se acercan.
- Si la distancia actual es menor, las partículas se separan.
- Si una partícula es fija, toda la corrección recae en la otra.

Relación

Esto permite pasar de una caída simple a un péndulo, una cadena o una malla usando exactamente la misma idea modular.

Actividad para clase:

Tenemos dos partículas unidas por una ligadura. La primera está fija. La segunda cae por gravedad.

- 1 Identifica qué parte del problema usa Euler.
- 2 Identifica qué parte usa corrección geométrica.
- 3 Explica por qué ambas cosas son necesarias para obtener un péndulo simple.

Respuesta esperada

Euler actualiza el movimiento. La ligadura impide que la distancia al punto fijo cambie. Juntas producen una oscilación restringida.

- Olvidar qué significa el paso temporal h .
- Cambiar signos sin revisar el sistema de coordenadas.
- No distinguir entre posición, velocidad y aceleración.
- Esperar precisión perfecta: Euler es sencillo, no exacto.

- 1 La física nos da una ecuación diferencial.
- 2 Introducimos variables auxiliares para obtener un sistema de primer orden.
- 3 Euler reemplaza derivadas por cambios discretos.
- 4 Esas fórmulas se convierten en un ciclo de actualización.
- 5 Luego añadimos restricciones para construir sistemas más complejos.

Entender este flujo permite reutilizar el método con muchas otras fuerzas físicas.

Una vez comprendida la estructura, se puede cambiar la aceleración para simular:

- viento,
- fricción,
- resortes,
- atracción o repulsión,
- movimiento orbital simplificado,
- cadenas y telas con múltiples ligaduras.